Report 1: live segmentation session, 27 Oct 2026

Snapshot taken at **16:44 PDT**, about 49 minutes into the acquisition. The acquisition and both segmenters were still running.

Summary

- **Acquisition is healthy.** LumaViewPro is imaging at a steady 1.65 images/s (one image every 0.60 s). 4,789 of 15,360 images (31%) are taken; the plate should finish around **18:30** (range 18:05–18:58).
- **Segmentation is correct but slower than the microscope.** The web station and the standalone script are each segmenting the full plate, sharing one GPU, at about 0.8 and 0.7 images/s. At this pace both finish around **22:00**, about 3.5 h after acquisition ends.
- **The station's masks match the script's exactly.** Every image both processed gave an identical mask. No failures and no GPU memory errors.
- **The workstation is within normal ranges on every measure.** The GPU is fully used, cool and under its power limit; CPU, RAM and disk have plenty of headroom.
- **Phase-contrast images find almost nothing.** PC images average 0.8 cells (max 21) against 121 for Green, yet take half the segmentation time.

The session

Capture folder	`ProtocolData/20261027_155550`
Started	15:55:50
Protocol	96-well plate, 4x Olympus, PC + Green, 2×2 tiles, 20 Z-slices: 15,360 images (160 per well)
Images	1900×1900, 8-bit TIFF
Segmentation settings	cpdino; diameter 45, flow 0.5, cellprob −0.2, min size 150 px, niter 500, ≥26 µm diameter filter, 1.878 µm/px; 2 images per GPU call
Running	LumaViewPro (71 min, including setup); standalone script `Etaluma_Live_Segmentation_Optimized (2).py` (44 min); web station (15 min since its last restart)

The machine

Host	`neil-linlab-GamingPC`
CPU	Intel Core Ultra 9 285K, 24 cores / 24 threads
RAM	62 GiB
GPU	NVIDIA GeForce RTX 5090, 32 GB VRAM, Blackwell (`sm_120`), 600 W power limit
NVIDIA driver	580.178.04 (CUDA 13.0)
Disk	Samsung 990 EVO Plus 2 TB NVMe, one volume for OS, captures and results (1.2 TB free)
OS	Ubuntu 24.04.5 LTS, kernel 7.0.0-38-generic
Microscope software	LumaViewPro 4.0.0-beta
Segmentation stack	Python 3.12.3, cellpose 4.2.1.1, torch 2.14.1+cu130, dinov3 (env `~/cellpose-cpdino-env`)

Progress

	Done	Of	Rate	Projection
Acquisition	4,789 images, 30 wells reached	15,360	1.65 img/s (99/min)	done ≈18:30 (18:05–18:58)
Station segmentation	718	15,360	0.80 img/s	≈22:00 if the script keeps running
Standalone script	1,929	15,360	0.72 img/s	≈22:00 if the station keeps running



Figure 1. Station Dashboard at 16:45. On the plate map, amber wells are being segmented, blue wells are acquired and waiting, and dark wells are not yet imaged.

The station started 30 minutes into the acquisition and is working through a backlog of 4,071 images. Its Dashboard reads **"Falling behind"** and **0/96 wells**. Both are expected: 0/96 counts only wells whose 160 images are *all* segmented, and no well is complete yet.

Workstation performance

The values below are from the station's 444 samples between 16:29 and 16:44, alongside the Dashboard's "What normal looks like" reading at 16:44.

What	Normal on this machine	16:44	Median	95th percentile	Max	Status
GPU load	80–100% while images wait	91%	95%	100%	100%	normal
GPU temperature	55–75 °C	70 °C	67 °C	73 °C	73 °C	normal
GPU power	400–600 W (limit 600 W)	538 W	451 W	588 W	599 W	normal
GPU memory	≈5.3 GB per segmenter, ≈10.7 GB for two	10.4 GB	10.4 GB	10.4 GB	10.4 GB	normal
CPU	10–25%	16%	16%	18%	89% (startup)	normal
RAM	≈20% of 62 GB	21%	21%	21%	22%	normal
Disk writes	5–50 MB/s	26 MB/s	22 MB/s	39 MB/s	48 MB/s	normal
Disk free	over 50 GB	1.2 TB				normal
Time between images	≈0.6 s	0.60 s	0.60 s	0.98 s	3.2 s (well move)	normal
Segmentation speed (station)	≈0.9 img/s while shared	0.81 img/s	0.80	0.93	1.00	normal

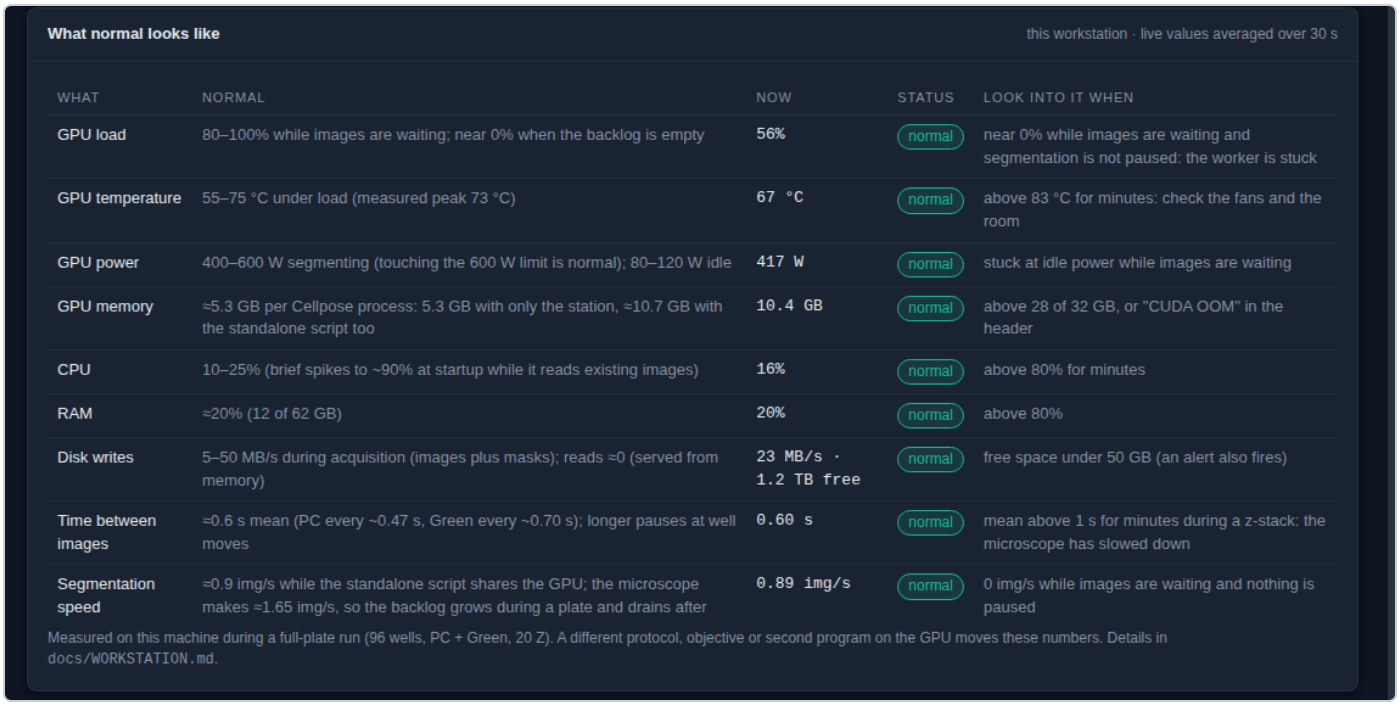


Figure 2. The Dashboard's "What normal looks like" card at 16:45. All nine measures read normal.



Figure 3. The last 16 minutes of station telemetry (16:29–16:45). Backlog rises steadily because acquisition outpaces segmentation. Throughput holds near 0.8 img/s. GPU load, temperature (≈67 °C) and power (400–600 W) stay high and steady. The gap between images stays flat at ≈0.6 s, so segmentation isn't slowing the microscope.

Reading it: the GPU is the only resource that's saturated, and it's running cool (73 °C peak) and below its power limit. CPU, RAM and disk are mostly idle, so segmentation isn't slowing the microscope down: the gap between images held at 0.60 s throughout.

Segmentation results so far (station, 718 images)

Channel	Images	Mean cells per image	Max	Images with 0 cells
Green	367	121	1,751	112 (31%)
PC	351	0.8	21	277 (79%)

The wells with the most cells in one Green image are A11 (1,751), B10 (1,706), A10 (1,645), B09 (1,563) and A04 (1,543). Counts are per image and are never summed over Z: the slices of one tile show the same cells.

Timing per image (station): median 1.08 s and 95th percentile 1.59 s, counting GPU work plus saving the mask, ROIs and outline. 359 batches, 0 failures, 0 GPU memory errors. For comparison, the script's last 60 batches took a median of 1.05 s per image, 0.86 s of it on the GPU.

Correctness: for every image both programs finished early in the session, the station's mask was byte-identical to the script's. The station also replaced the script's slowest step (outlining every cell) with an equivalent that takes 0.2 s instead of up to 12 s on a crowded image, with identical output.

Why segmentation can't keep up, and options

The microscope produces 1.65 images/s. Two copies of the same segmentation share one GPU, so each gets about half of it.

Option	Expected effect	Trade-off
Stop the standalone script (Ctrl+C; it can resume later)	Station roughly doubles to ≈1.2–1.6 img/s, close to the acquisition rate	One set of outputs instead of two
Stop segmenting PC images	Halves the work; both programs could keep up	Loses PC masks, which currently find almost no cells
Segment one in-focus Z per tile instead of all 20	~20× less work	Changes what is measured; needs validation against the current results

These are science decisions for the lab; no settings were changed.

Where the results are

- Station: `~/Desktop/RP/Station Output/20261027_155550/`, containing `masks/`, `rois/`, `debug_plots/` and `density_per_well.csv`
- Script: `~/Desktop/RP/Mock Test Output/` (same layout)
- Live view: http://localhost:8100 (Dashboard and the run page)
- Machine specs and normal ranges: `docs/WORKSTATION.md`